UNIT-1

Finite Automata

1.Structural Representation of Finite automata

Finite automata are abstract machines with finite states, processing inputs via transitions. They are represented structurally as deterministic finite automata (DFA) or nondeterministic finite automata (NFA), each with distinct characteristics.

- **Formal Definition and Representations:**
  - Both DFA and NFA are defined as 5-tuples: $M = (Q, \Sigma, \delta, q_0, F)$, where:
    - Q: Finite, non-empty set of states.
    - Σ: Finite, non-empty set of input symbols (alphabet).
    - δ: Transition function, mapping $Q \times \Sigma$ to Q for DFA, or to 2^Q for NFA, potentially including ε (null) transitions.
    - q_0: Initial state, $q_0 \in Q$.
    - F: Subset of Q, final (accepting) states.
  - **State Diagrams:** Graphical, with nodes as states, edges as transitions labeled by input symbols, initial state marked by an arrow, and final states by double circles.
  - **Transition Tables:** Tabular, with rows for states, columns for input symbols, and cells for next state(s). For example, a DFA for strings ending with 'a' over $\Sigma = \{a, b\}$ has:

State \ Symbol	a	b
q0	q1	q0
q1	q1	q0

    - Here, q_0 is initial, q_1 is final, and transitions are deterministic.
- **NFA Representation Example:**
  - For the same language, an NFA might have:

State \ Symbol	a	b
q0	{q0, q1}	q0
q1	φ	φ

  - NFA allows multiple transitions, like $\delta(q_0, a) = \{q_0, q_1\}$, and can include ε-transitions, making design more flexible but potentially less intuitive.

DeeperSearch ×

-

2.Automata and Complexity

Automata theory is intrinsically linked to computational complexity, exploring what can be computed and how efficiently, using models like finite automata, pushdown automata, and Turing machines.

- **Automata and Computational Power:**
  - Automata classify languages in the Chomsky hierarchy:
    - **Finite Automata (DFA/NFA):** Recognize regular languages, processed in linear time.
    - **Pushdown Automata:** Recognize context-free languages, requiring stack memory.
    - **Turing Machines:** Recognize recursively enumerable languages, with unbounded memory.
  - The hierarchy reflects increasing computational power, with finite automata at the base, handling patterns like strings ending with 'a'.
- **Complexity Theory Integration:**
  - Complexity theory, developed in the 1960s, studies time and memory resources. For instance, Shannon's 1956 work provided complexity measures for Turing machines, while the Myhill–Nerode theorem (1958) gives conditions for regular languages, linking state count to complexity
  - Finite automata operations, like string acceptance, are typically linear time, but NFA to DFA conversion can exponentially increase states, impacting efficiency.
- **Historical Context:**
  - Automata theory emerged mid-20th century, with "Automata Studies" (1956) by Shannon et al. introducing regular languages and complexity measures. Complexity theory evolved with Hartmanis and Stearns' 1964 work on recursive sequences, shaping modern understanding .

3.Central Concept of Automata Theory

Symbol: Symbol is any character which is meaningful or meaningless.
Example: A,B,C,….,Z
a,b,c,…z
0,1,2,…9
∑, λ, δ, π…

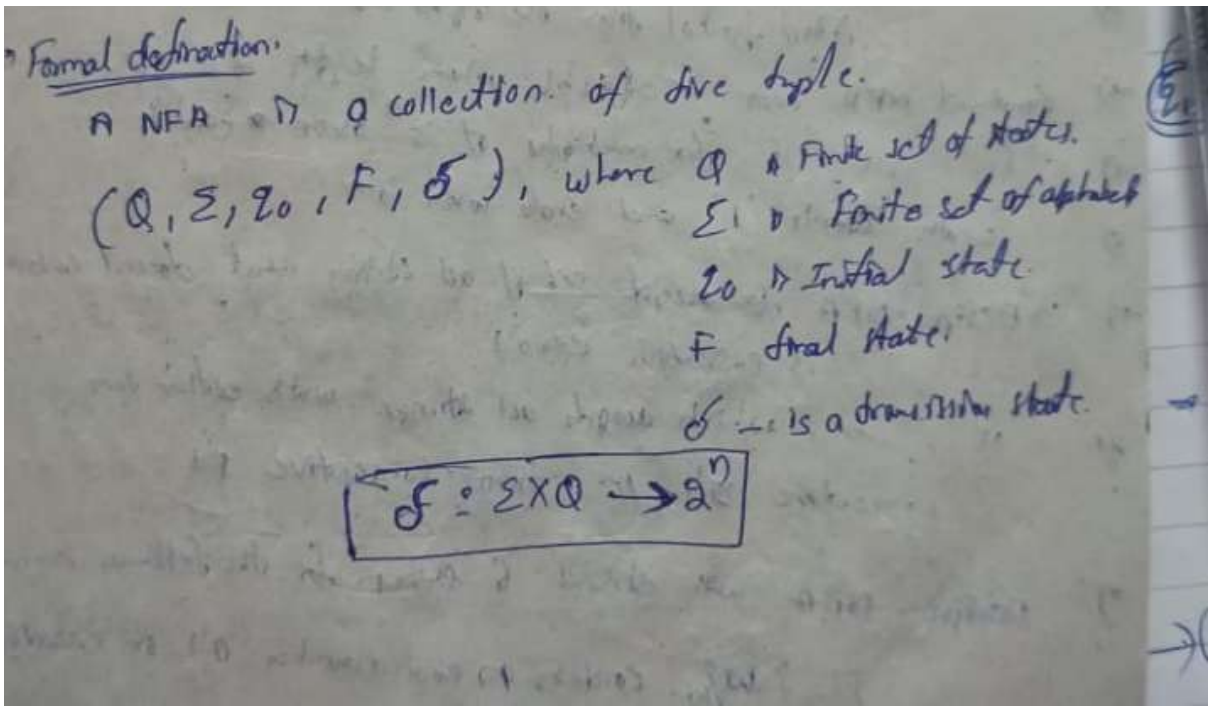
Alphabet: An alphabet is a finite, and non empty set of symbols. It is denoted by ∑.
Common alphabets include:
Example: 1. ∑= {0, 1}, is the binary alphabet.
2. ∑ = {a, b,…, z}, is lower-case English alphabet.
3. ∑ = {0,1,2,…9}, is decimal number alphabet.

String: A string is a finite sequence of symbols from some alphabet.
Example: ∑= {0, 1}, is the binary alphabet.
Strings are 00,01,10,11, ε …

- **Languages:**
  - A language is a subset of Σ^*, the set of all finite strings over Σ. Example: The language of all strings ending with 'a' is $L = \{a, ba, aa, bba, \ldots\}$, recognized by both DFA and NFA .
- **Problems:**
  - Problems involve tasks like determining if a string belongs to a language, designing automata, or converting representations. Applications include text processing, compilers, and network protocols, where automata recognize patterns efficiently.

Non-Deterministic Finite Automata

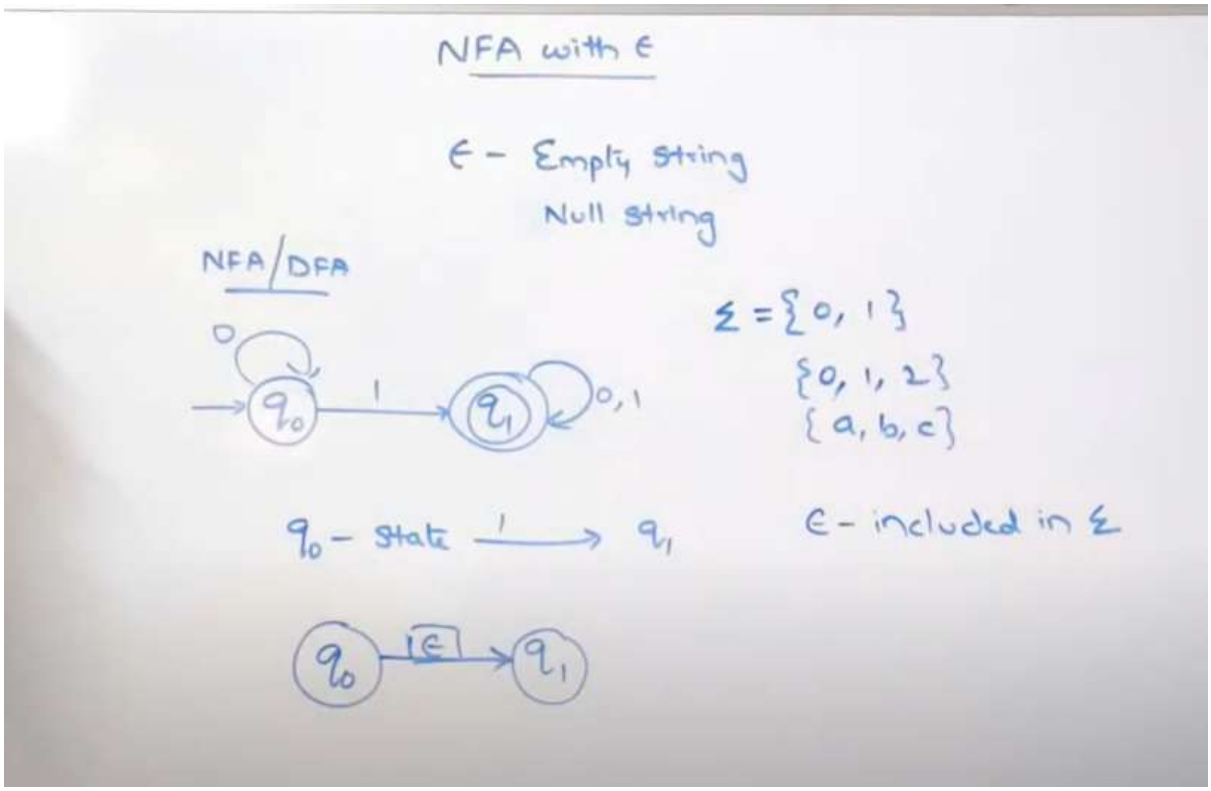
4.Formal Defination of NFA

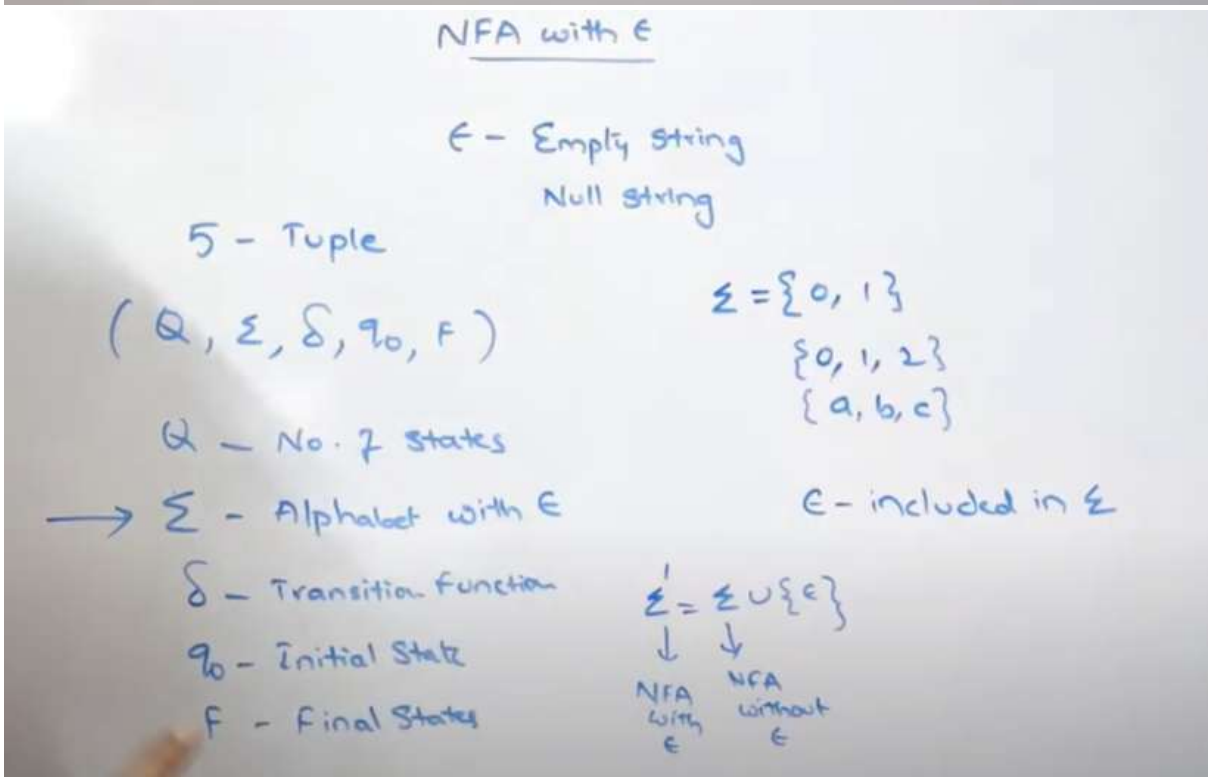


Application

- **Text Search and Pattern Matching**: NFAs efficiently model patterns for searching substrings or regular expressions in text, such as finding "ab" in a string, due to their ability to handle multiple transitions.
- **Lexical Analysis in Compilers**: NFAs are used to tokenize source code by recognizing patterns like identifiers or keywords, enabling efficient lexical scanner design.
- **Regular Expression Implementation**: NFAs directly represent regular expressions, allowing tools like grep or regex libraries to match complex patterns concisely.
- **Network Protocol Analysis**: NFAs model protocols with nondeterministic behavior, such as packet sequences, aiding in validation and simulation.
- **Natural Language Processing**: NFAs assist in processing ambiguous or context-sensitive language structures, like parsing partial matches in speech recognition.
- **Software Verification**: NFAs model system behaviors with nondeterministic paths, used in model checking to verify software correctness.

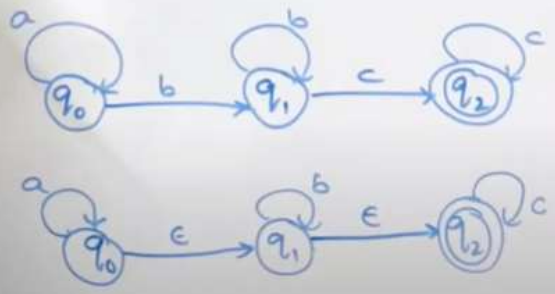
5. Non Finite Automata with Epsilon-Transitions





NFA with ϵ

→ Construct NFA with ϵ which accept all strings over alphabet $\Sigma = \{a, b, c\}$ where string contains multiple a's followed by multiple b's followed by multiple c's.



$\Sigma = \{a, b, c, \epsilon\}$

	a	b	c	ϵ
q_0	q_0	ϕ	ϕ	q_1
q_1	ϕ	q_1	ϕ	q_2
q_2	ϕ	ϕ	q_2	ϕ

6.Deterministic Finite Automata

In DFA, for each input symbol, one can determine the state to which the machine will move. Hence, it is called Deterministic Automaton. As it has a finite number of states, the machine is called Deterministic

Finite Machine or Deterministic Finite Automaton.

Formal Definition of a DFA

A DFA can be represented by a 5-tuple (Q, ∑, δ, q0, F) where −

• Q is a finite set of states.

• ∑ is a finite set of symbols called the alphabet.

• δ is the transition function where δ: Q × ∑ → Q

• q0 is the initial state from where any input is processed (q0 ∈ Q).

• F is a set of final state/states of Q (F ⊆ Q).

Graphical Representation of a DFA

A DFA is represented by digraphs called state diagram.

• The vertices represent the states.

• The arcs labeled with an input alphabet show the transitions.

• The initial state is denoted by an empty single incoming arc.

• The final state is indicated by double circles.

7.The Language of DFA

The Language of DFA

- A DFA is a five-tuple: $M = (Q, \Sigma, \delta, q_0, F)$

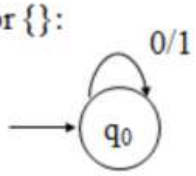
Q	A finite set of states
Σ	A finite input alphabet
q_0	The initial/starting state, q_0 is in Q
F	A set of final/accepting states, which is a subset of Q
δ	A transition function, which is a total function from Q x Σ to Q

$\delta: (Q \times \Sigma) \rightarrow Q$ δ is defined for any q in Q and s in Σ, and $\delta(q,s) = q'$ is equal to another state q' in Q.

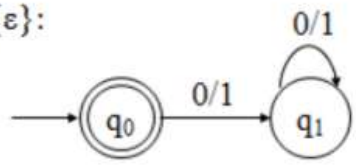
Intuitively, $\delta(q,s)$ is the state entered by M after reading symbol s while in state q.

- Let $\Sigma = \{0, 1\}$. Give DFAs for $\{\}$, $\{\varepsilon\}$, Σ^*, and Σ^+.

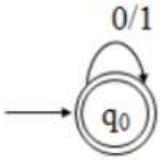
For {}:



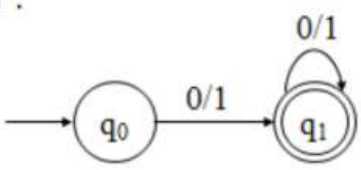
For {ε}:



For Σ^*:



For Σ^+:

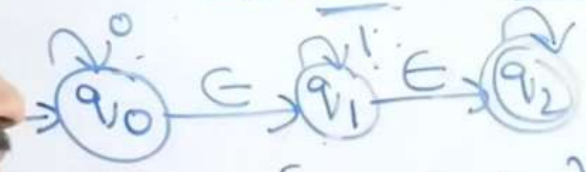


8. Conversion of NFA with €-transitions to NFA without €-transitions

Converting NFA with ϵ-moves to NFA without ϵ-moves

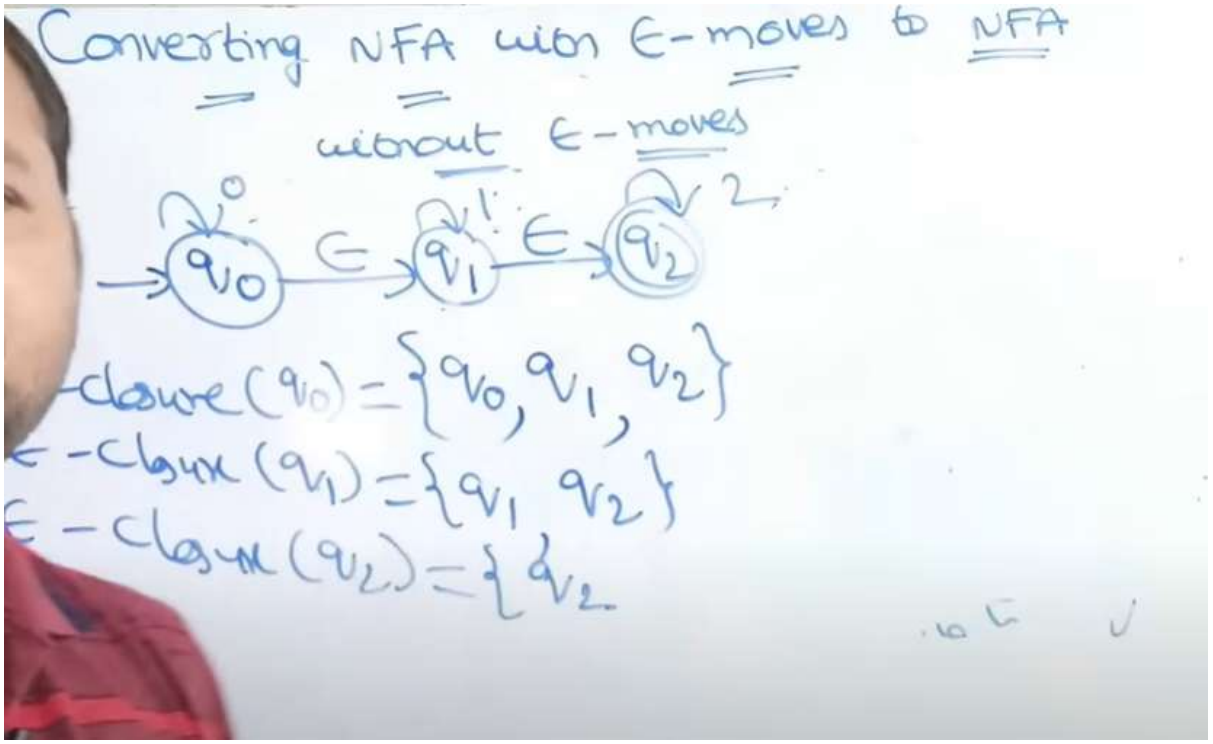
1. Calculate ϵ-closure of all states.
2. Calculate extended transition function $\hat{\delta}'(q_0, 0)$

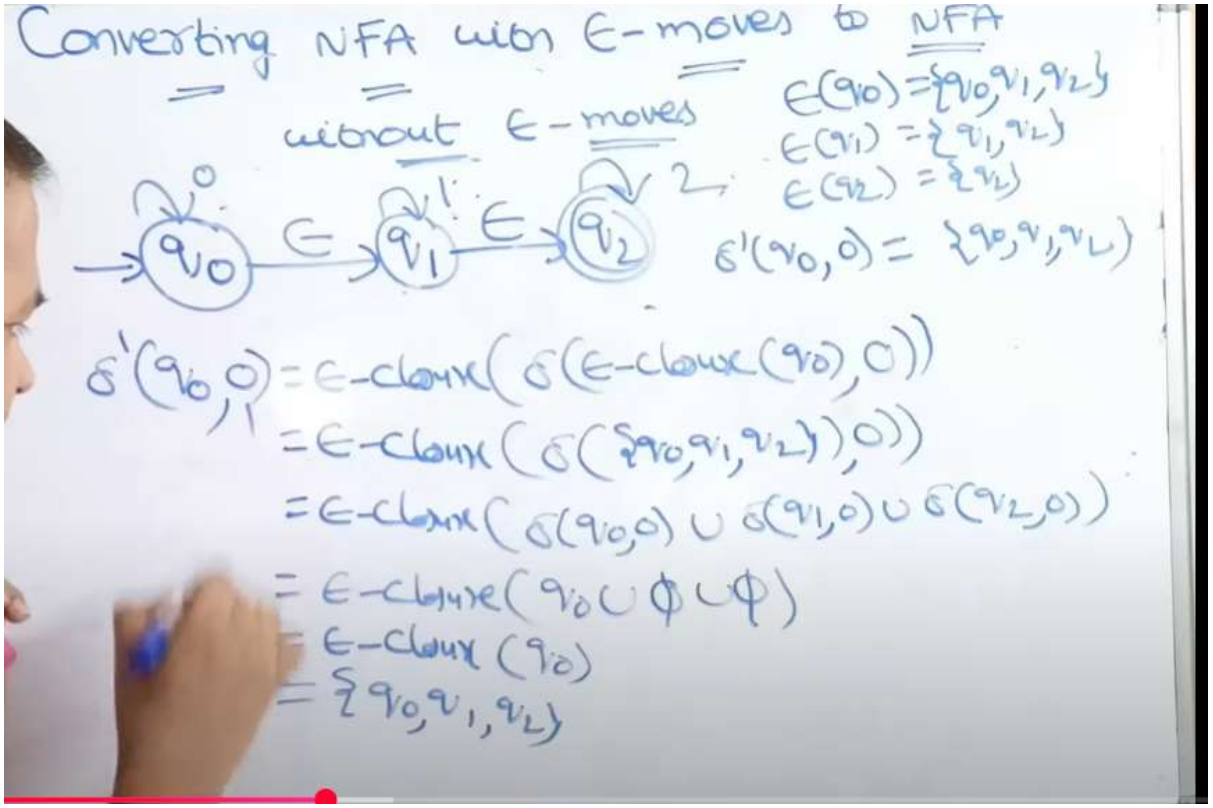
Converting NFA with ϵ-moves to NFA without ϵ-moves



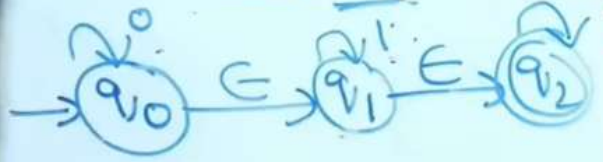
closure(q_0) = $\{q_0, q_1, q_2\}$

ϵ-closure(q) : set of states that are reachable from state q with ϵ-transition.


Converting NFA with ϵ-moves to NFA
without ϵ-moves
0
1
2
q_0
q_1
q_2
ϵ
ϵ
ϵ-closure(q_0) = {q_0, q_1, q_2}
ϵ-closure(q_1) = {q_1, q_2}
ϵ-closure(q_2) = {q_2}


Converting NFA with ϵ-moves to NFA
without ϵ-moves
0
1
2
q_0
q_1
q_2
ϵ
ϵ
ϵ(q_0) = {q_0, q_1, q_2}
ϵ(q_1) = {q_1, q_2}
ϵ(q_2) = {q_2}
δ'(q_0, 0) = {q_0, q_1, q_2}
δ'(q_0, 0) = ϵ-closure(δ(ϵ-closure(q_0), 0))
= ϵ-closure(δ({q_0, q_1, q_2}, 0))
= ϵ-closure(δ(q_0, 0) ∪ δ(q_1, 0) ∪ δ(q_2, 0))
= ϵ-closure(q_0 ∪ ϕ ∪ ϕ)
= ϵ-closure(q_0)
= {q_0, q_1, q_2}

without ϵ-moves



$\epsilon(q_1) = \{q_1, q_2\}$

$\epsilon(q_2) = \{q_2\}$

$\delta'(q_0, 0) = \{q_0, q_1, q_2\}$

$\delta'(q_0, 1) = \{q_1, q_2\}$

$\delta'(q_0, 1) = \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_0), 1))$

$= \epsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, 1))$

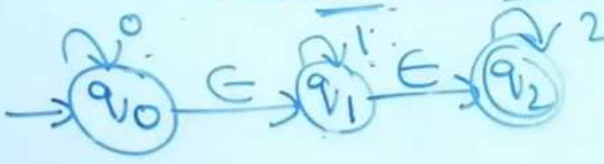
$= \epsilon\text{-closure}(\delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1))$

$= \epsilon\text{-closure}(\phi \cup q_1 \cup \phi)$

$= \{q_1, q_2\}$

Converting NFA with ϵ-moves to NFA

without ϵ-moves



$\epsilon(q_0) = \{q_0, q_1, q_2\}$

$\epsilon(q_1) = \{q_1, q_2\}$

$\epsilon(q_2) = \{q_2\}$

$\delta'(q_0, 0) = \{q_0, q_1, q_2\}$

$\delta'(q_0, 1) = \{q_1, q_2\}$

$\delta'(q_0, 2) = \{q_2\}$

$\delta'(q_0, 2) = \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_0), 2))$

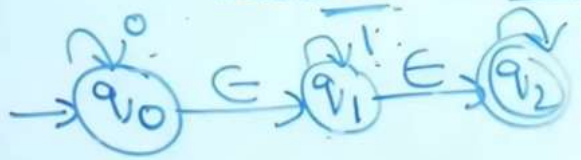
$= \epsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, 2))$

$= \epsilon\text{-closure}(\delta(q_0, 2) \cup \delta(q_1, 2) \cup \delta(q_2, 2))$

$= \epsilon\text{-closure}(\phi \cup \phi \cup q_2)$

$= \{q_2\}$

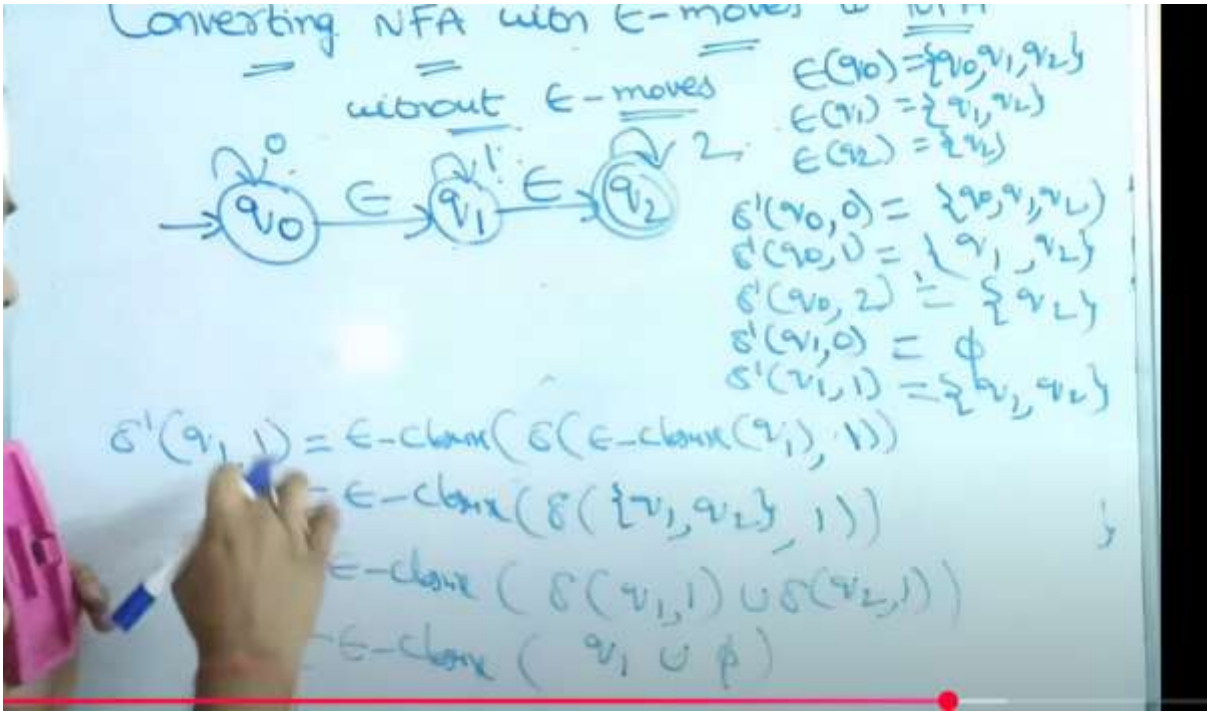
Converting NFA with ε-moves to NFA without ε-moves



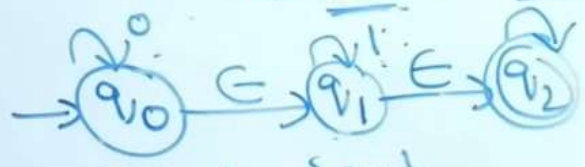
$\epsilon(q_0) = \{q_0, q_1, q_2\}$
$\epsilon(q_1) = \{q_1, q_2\}$
$\epsilon(q_2) = \{q_2\}$

$\delta'(q_0, 0) = \{q_0, q_1, q_2\}$
$\delta'(q_0, 1) = \{q_1, q_2\}$
$\delta'(q_0, 2) = \{q_2\}$
$\delta'(q_1, 0) = \phi$

$\delta'(q_1, 0) = \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_1), 0))$
$= \epsilon\text{-closure}(\delta(\{q_1, q_2\}, 0))$
$= \epsilon\text{-closure}(\delta(q_1, 0) \cup \delta(q_2, 0))$
$= \epsilon\text{-closure}(\phi \cup \phi)$



Converting NFA with ϵ-moves to NFA without ϵ-moves



$\delta'(q_1, 2) = \{q_2\}$

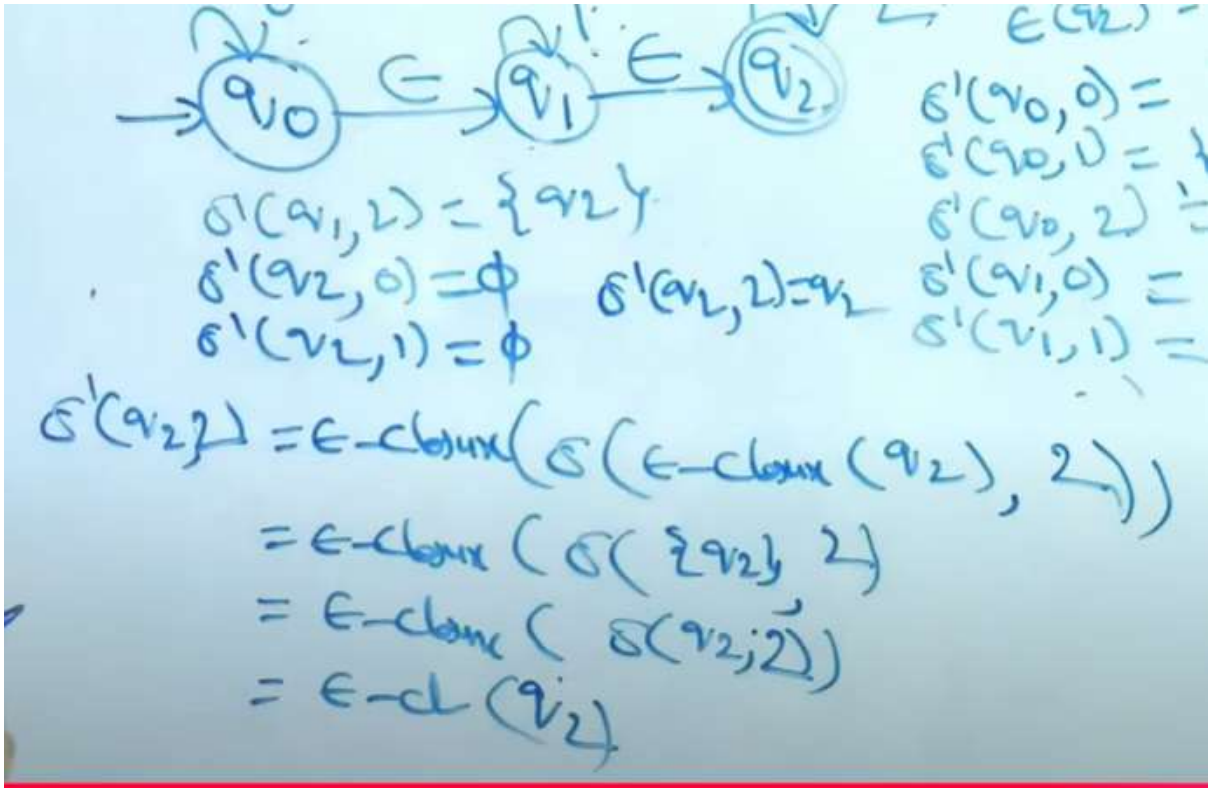
$\epsilon(q_0) = \{q_0, q_1, q_2\}$
$\epsilon(q_1) = \{q_1, q_2\}$
$\epsilon(q_2) = \{q_2\}$

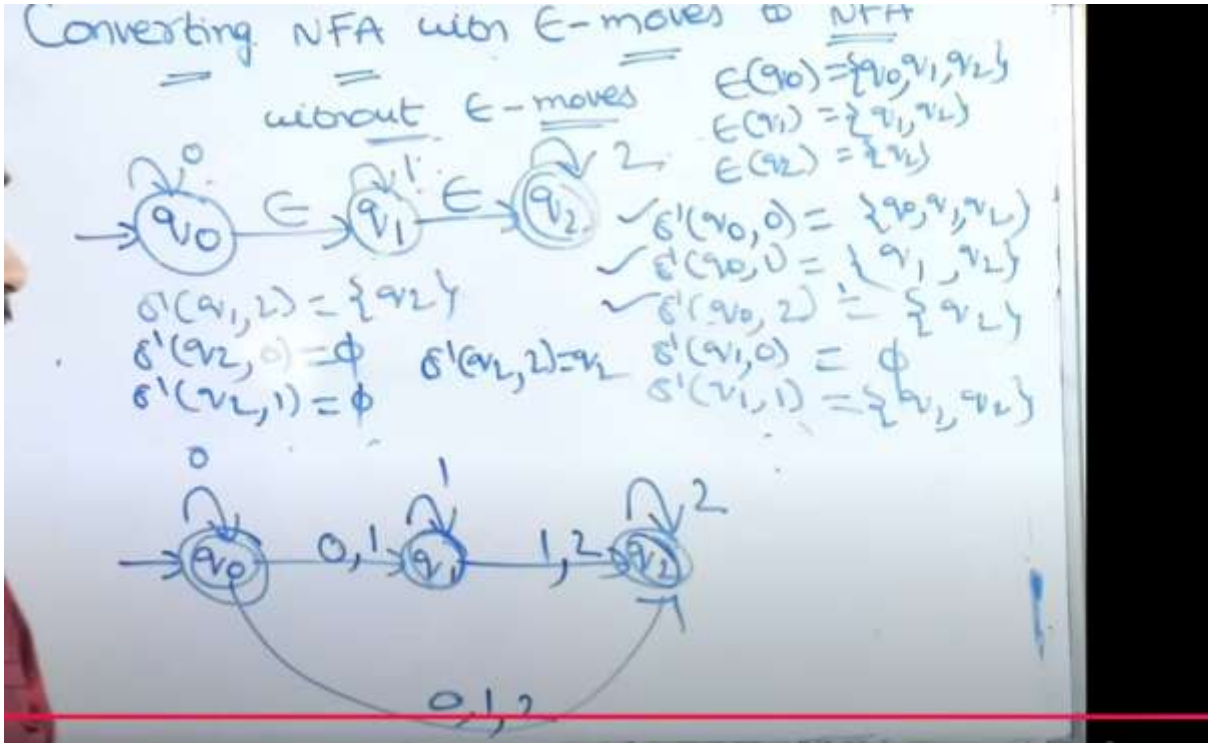
$\delta'(q_0, 0) = \{q_0, q_1, q_2\}$
$\delta'(q_0, 1) = \{q_1, q_2\}$
$\delta'(q_0, 2) = \{q_2\}$
$\delta'(q_1, 0) = \phi$
$\delta'(q_1, 1) = \{q_1, q_2\}$

$\delta'(q_1, 2) = \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_1), 2))$
$= \epsilon\text{-closure}(\delta(\{q_1, q_2\}, 2))$
$= \epsilon\text{-closure}(\delta(q_1, 2) \cup \delta(q_2, 2))$
$= \epsilon\text{-closure}(\phi \cup q_2)$

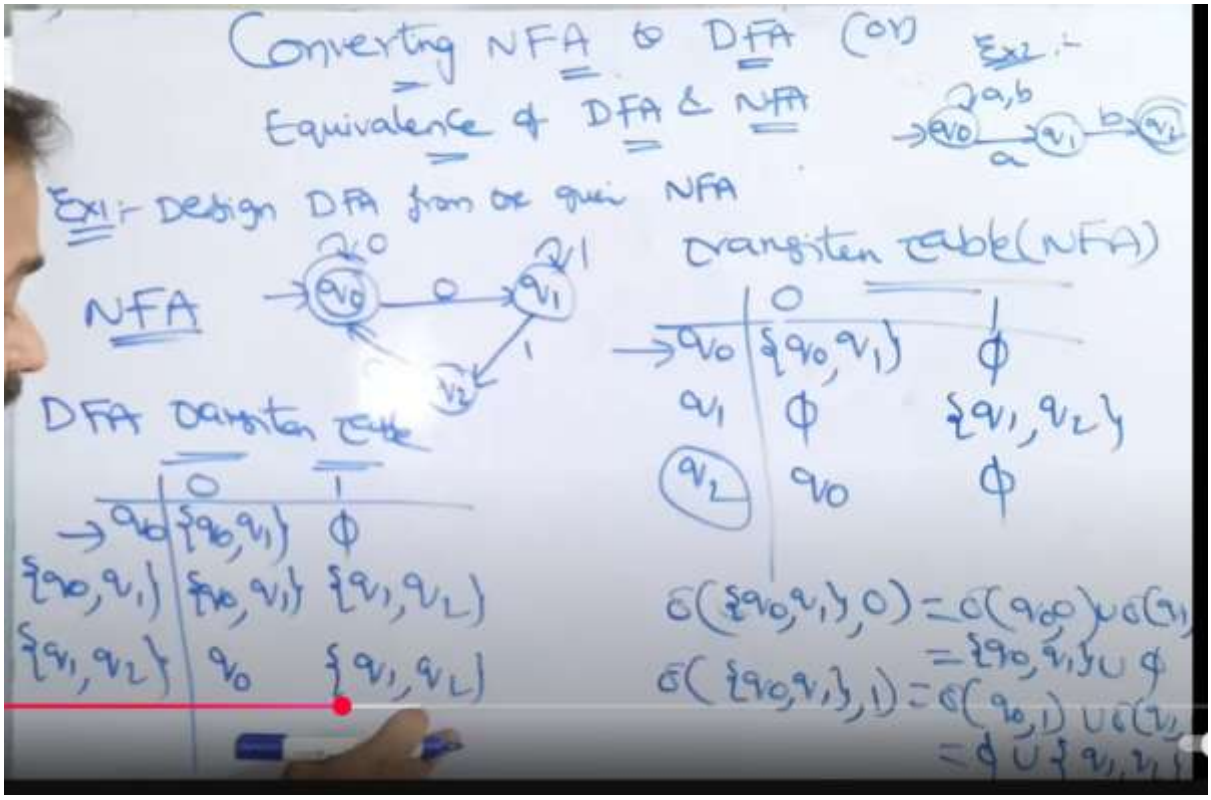
$\delta'(q_1, 1) = \{$

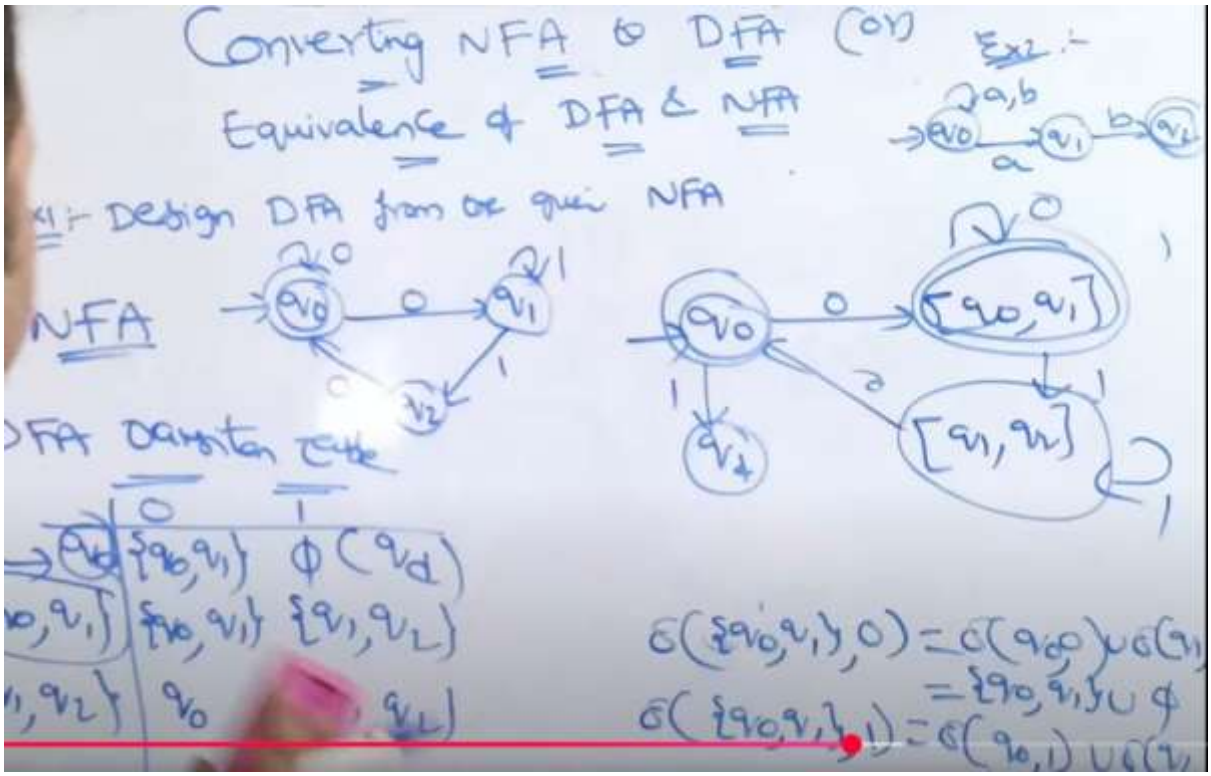
$\delta'(q_2, 1) = \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_2), 1))$
$= \epsilon\text{-closure}(\delta(\{q_2\}, 1))$
$= \epsilon\text{-closure}(\delta(q_2, 1))$
$= \epsilon\text{-cl}(\phi)$


q_0
q_1
q_2
ϵ
ϵ
$\delta'(q_1, 2) = \{q_2\}$
$\delta'(q_2, 0) = \phi$
$\delta'(q_2, 1) = \phi$
$\delta'(q_2, 2) = q_2$
$\delta'(q_0, 0) =$
$\delta'(q_0, 1) =$
$\delta'(q_0, 2) =$
$\delta'(q_1, 0) =$
$\delta'(q_1, 1) =$
$\epsilon(q_2) =$
$\delta'(q_2, 2) = \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_2), 2))$
$= \epsilon\text{-closure}(\delta(\{q_2\}, 2))$
$= \epsilon\text{-closure}(\delta(q_2, 2))$
$= \epsilon\text{-cl}(q_2)$


Converting NFA with ϵ-moves to NFA without ϵ-moves
q_0
q_1
q_2
ϵ
ϵ
$\epsilon(q_0) = \{q_0, q_1, q_2\}$
$\epsilon(q_1) = \{q_1, q_2\}$
$\epsilon(q_2) = \{q_2\}$
$\delta'(q_1, 2) = \{q_2\}$
$\delta'(q_2, 0) = \phi$
$\delta'(q_2, 1) = \phi$
$\delta'(q_2, 2) = q_2$
$\delta'(q_0, 0) = \{q_0, q_1, q_2\}$
$\delta'(q_0, 1) = \{q_1, q_2\}$
$\delta'(q_0, 2) = \{q_2\}$
$\delta'(q_1, 0) = \phi$
$\delta'(q_1, 1) = \{q_1, q_2\}$
0
1
2
0, 1
1, 2
0, 1, 2

9. . Conversion of NFA to DFA





10.Moore and Melay machine

Moore and Mealy Machines are Transducers that help in producing outputs based on the input of the current state or previous state

Moore Machines

Moore Machines are finite state machines with output value and its output depends only on the present state. It can be defined as (Q, q0, ∑, O, δ, λ) where:

- *Q is a finite set of states.*
- *q0 is the initial state.*
- *∑ is the input alphabet.*
- *O is the output alphabet.*
- *δ is the transition function which maps Q×∑ → Q.*
- *λ is the output function which maps Q → O.*

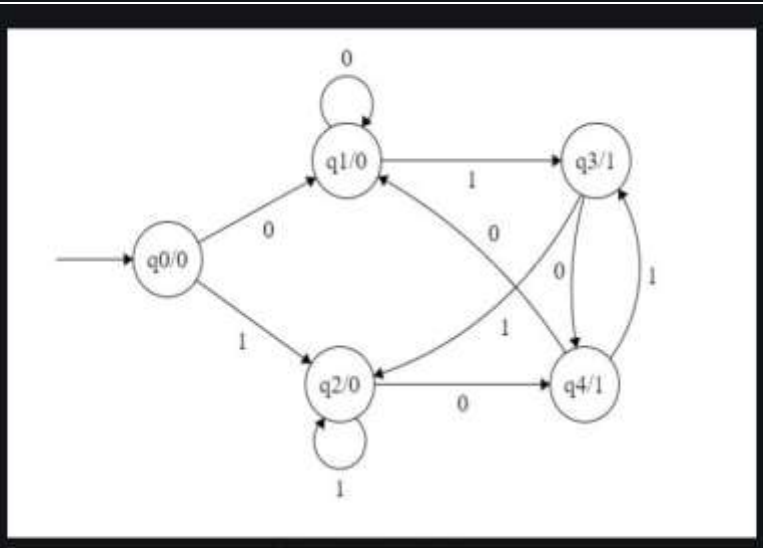


Figure 1: Moore Machines

In the Moore machine shown in Figure 1, the output is represented with each input state separated by /. The length of output for a Moore Machine is greater than input by 1.

- **Input:** 1,1
- **Transition:** δ (q0,1,1)=> δ(q2,1)=>q2
- **Output:** 000 (0 for q0, 0 for q2 and again 0 for q2)

Mealy Machines

Mealy machines are also finite state machines with output value and their output depends on the present state and current input symbol. It can be defined as (Q, q0, ∑, O, δ, λ') where:

- *Q is a finite set of states.*
- *q0 is the initial state.*
- *∑ is the input alphabet.*
- *O is the output alphabet.*
- *δ is the transition function which maps Q×∑ → Q.*
- *'λ' is the output function that maps Q×∑→ O.*

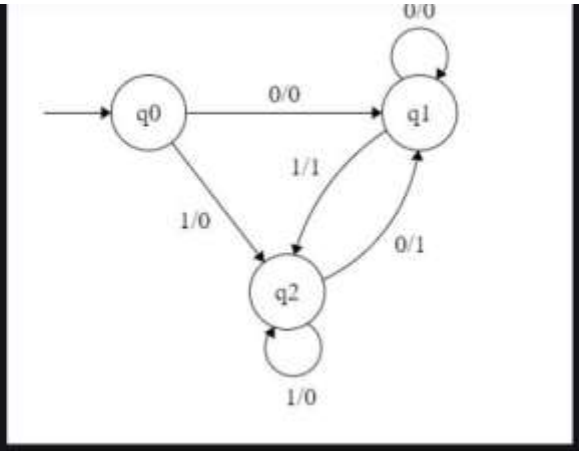


Figure 2:Mealy Machines

In the mealy machine shown in Figure 1, the output is represented with each input symbol for each state separated by /. The length of output for a mealy machine is equal to the length of input.

- **Input:** 1,1
- **Transition:** δ (q0,1,1)=> δ(q2,1)=>q2
- **Output:** 00 (q0 to q2 transition has Output 0 and q2 to q2 transition also has Output 0)

UNIT-2

Regular Expression

1.Finite Automata and Regular Expression

If L=L(A) for some DFA, then there is a regular expression R such that L=L(R)

We are going to construct regular expressions from a DFA by eliminating states.

When we eliminate a state s, all the paths that went through s no longer exist in the automaton.

If the language of the automaton is not to change, we must include, on an arc

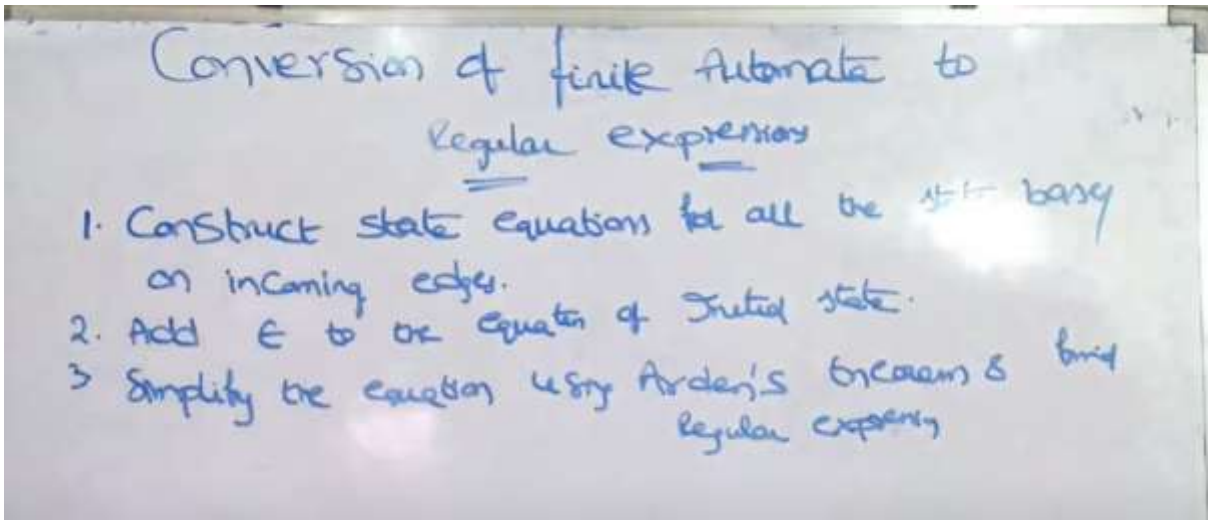
that goes directly from q to p ,the labels of paths that went from some state q to state p , through s.

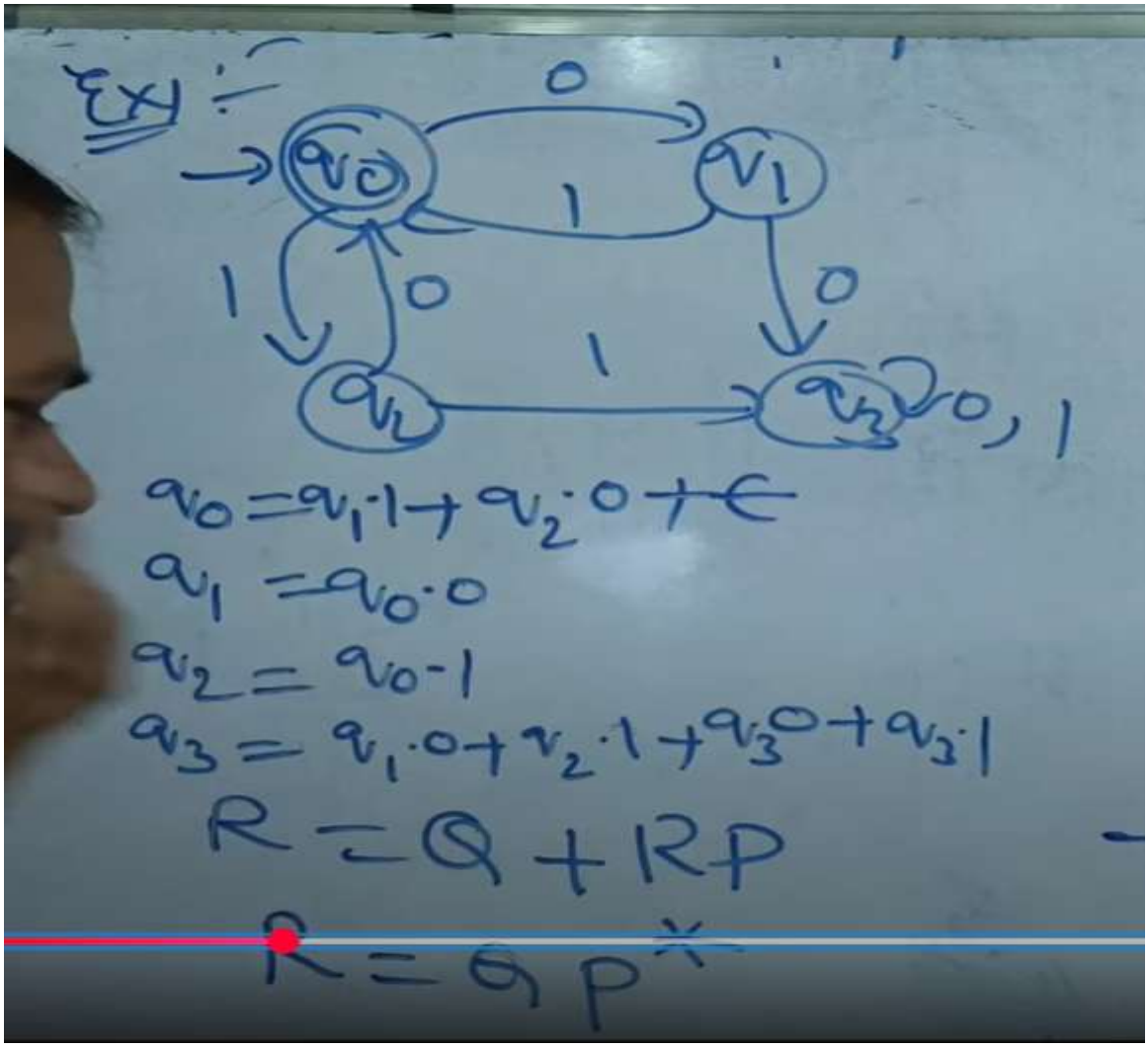
The label of this arc can now involve strings, rather than single symbols (may be an infinite number of strings).

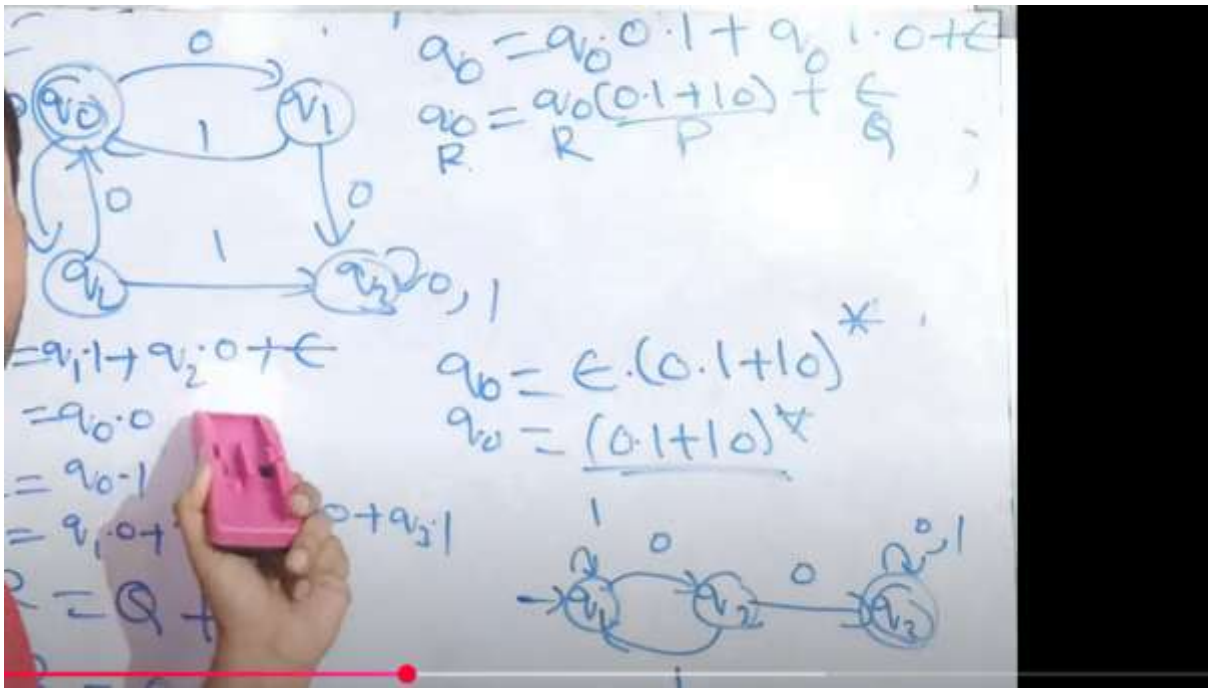
We use a regular expression to represent all such strings.

Thus, we consider automata that have regular expressions as labels.

2.Construct finite automata to regular expression




Ex :-
q_0
q_1
q_2
q_3
0
1
0, 1
$q_0 = q_1 \cdot 1 + q_2 \cdot 0 + \epsilon$
$q_1 = q_0 \cdot 0$
$q_2 = q_0 \cdot 1$
$q_3 = q_1 \cdot 0 + q_2 \cdot 1 + q_3 0 + q_3 \cdot 1$
$R = Q + RP$
$R = QP^*$


$q_0 = q_0 \cdot 0 \cdot 1 + q_0 \cdot 1 \cdot 0 + \epsilon$
$q_0 = q_0(0 \cdot 1 + 1 \cdot 0) + \epsilon$
R R P Q
$q_0 = \epsilon \cdot (0 \cdot 1 + 1 \cdot 0)^*$
$q_0 = (0 \cdot 1 + 1 \cdot 0)^*$

3.Application of Regular Expression

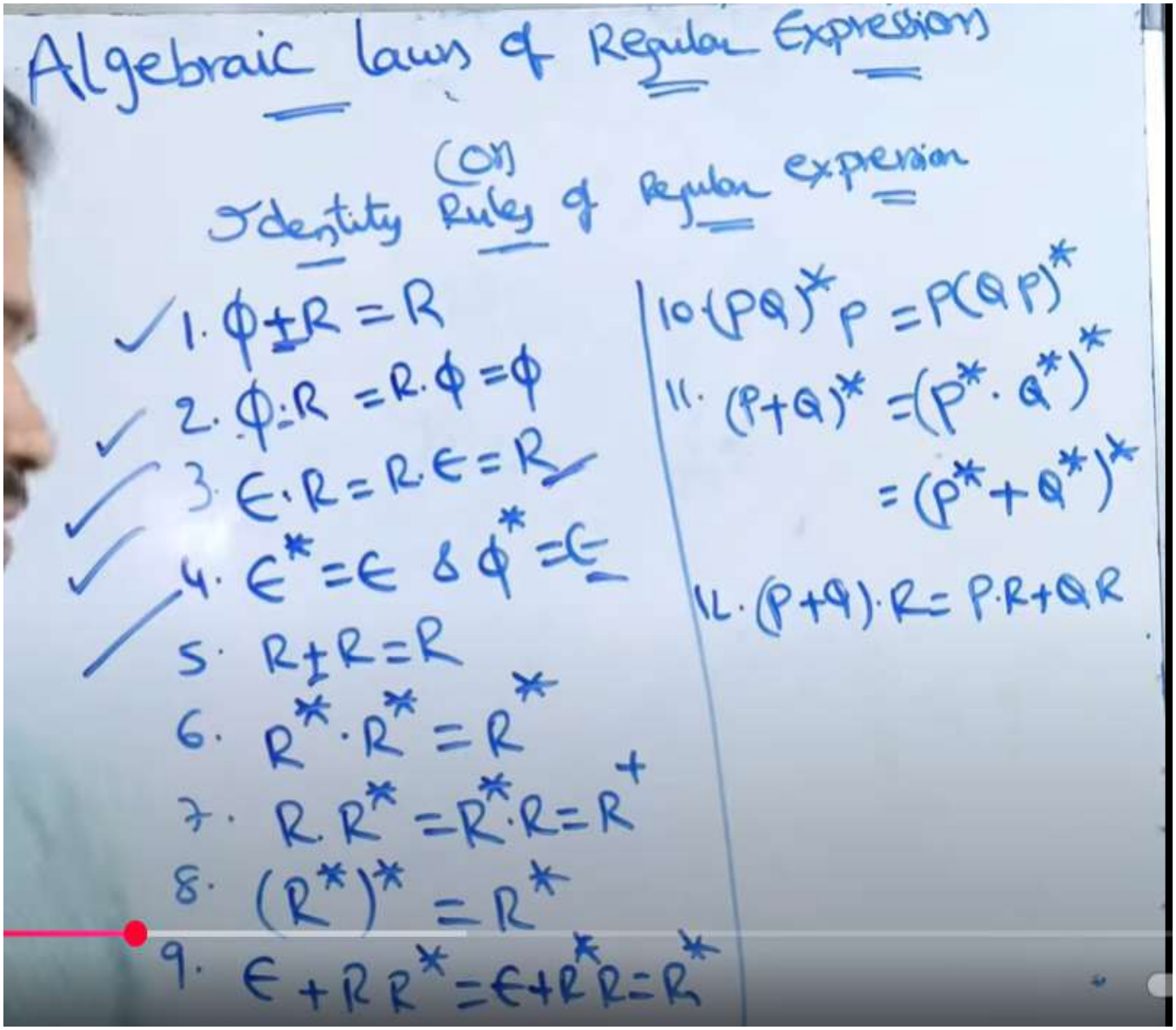
- Data Validation
- Data Scraping
- Data Wrangling
- Simple Parsing
- Syntax Highlighting Systems
- Internet Search Engines

4.Algebraic Laws for Regular Expression

Definition :Two regular expressions with variables are equivalent if whatever

languageswe substitute for the variables, the results of the two expressions are the

same language.



5.Pumping Lemma for Regular Expression

Pumping Lemma (For Regular Languages)

>> Pumping Lemma is used to prove that a Language is NOT REGULAR

>> It cannot be used to prove that a Language is Regular

If A is a Regular Language, then A has a Pumping Length 'P' such that any string 'S' where $|S| \geq P$ may be divided into 3 parts $S = x\,y\,z$ such that the following conditions must be true:

(1) $x\,y^i z \in A$ for every $i \geq 0$
(2) $|y| > 0$
(3) $|xy| \leq P$

To prove that a language is not Regular using PUMPING LEMMA, follow the below steps:

(We prove using Contradiction)

-> Assume that A is Regular
-> It has to have a Pumping Length (say P)
-> All strings longer than P can be pumped $|S| \geq P$
-> Now find a string 'S' in A such that $|S| \geq P$
-> Divide S into x y z
-> Show that $x\,y^i z \notin A$ for some i
-> Then consider all ways that S can be divided into x y z
-> Show that none of these can satisfy all the 3 pumping conditions at the same time
-> S cannot be Pumped == CONTRADICTION

Pumping Lemma (For Regular Languages) - EXAMPLE (Part-1)

Using Pumping Lemma prove that the language $A = \{a^n b^n \mid n \geq 0\}$ is Not Regular

Proof:

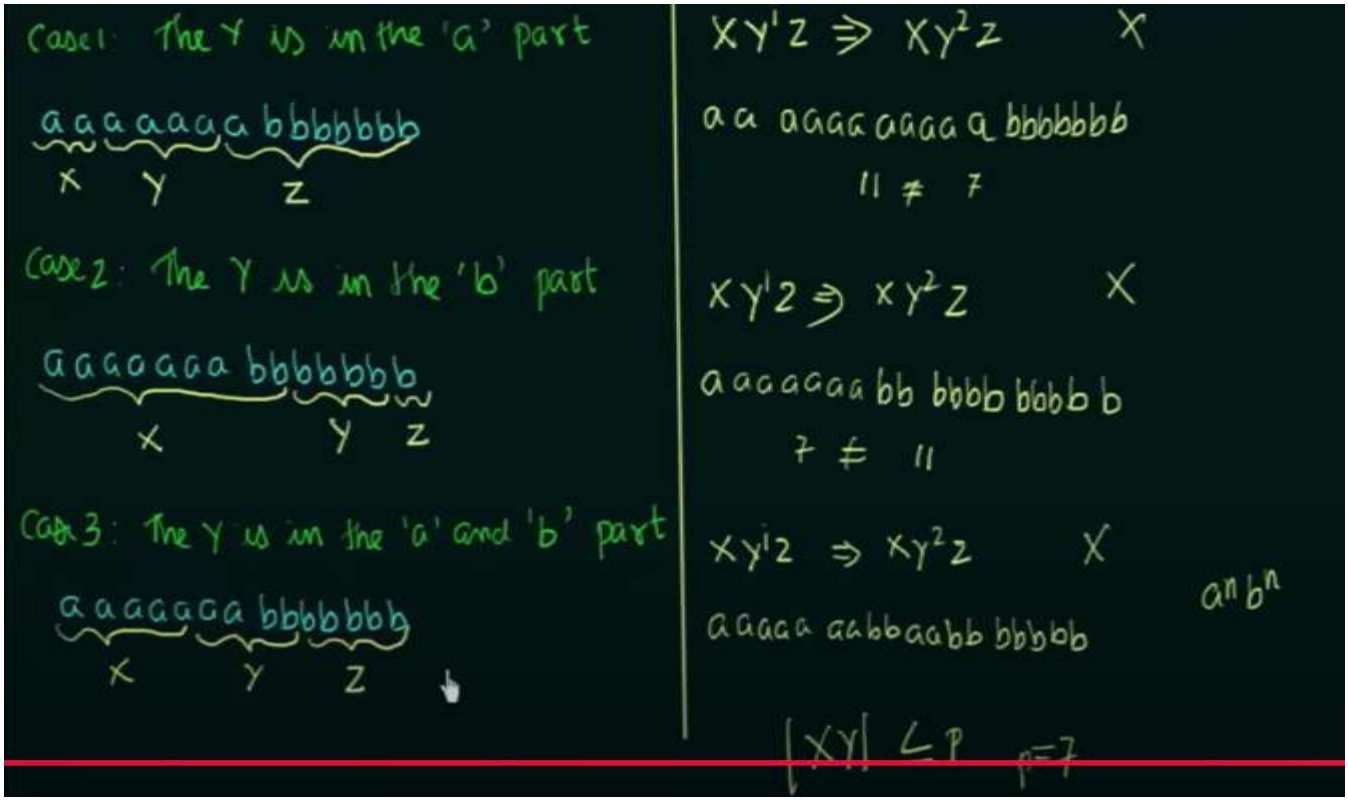
Assume that A is Regular

Pumping length = P

$S = a^P b^P \Rightarrow S = aaaaaaaabbbbbbbb$

x y z

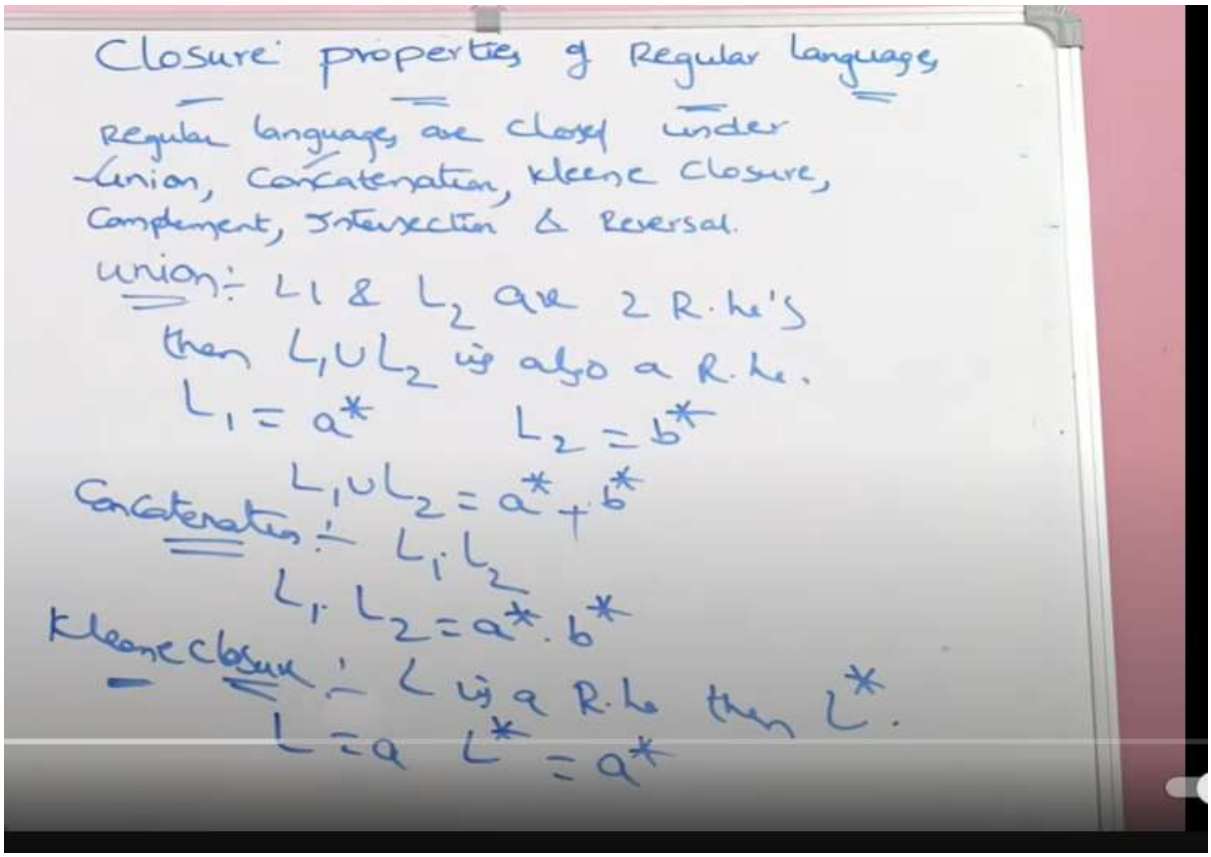
P = 7

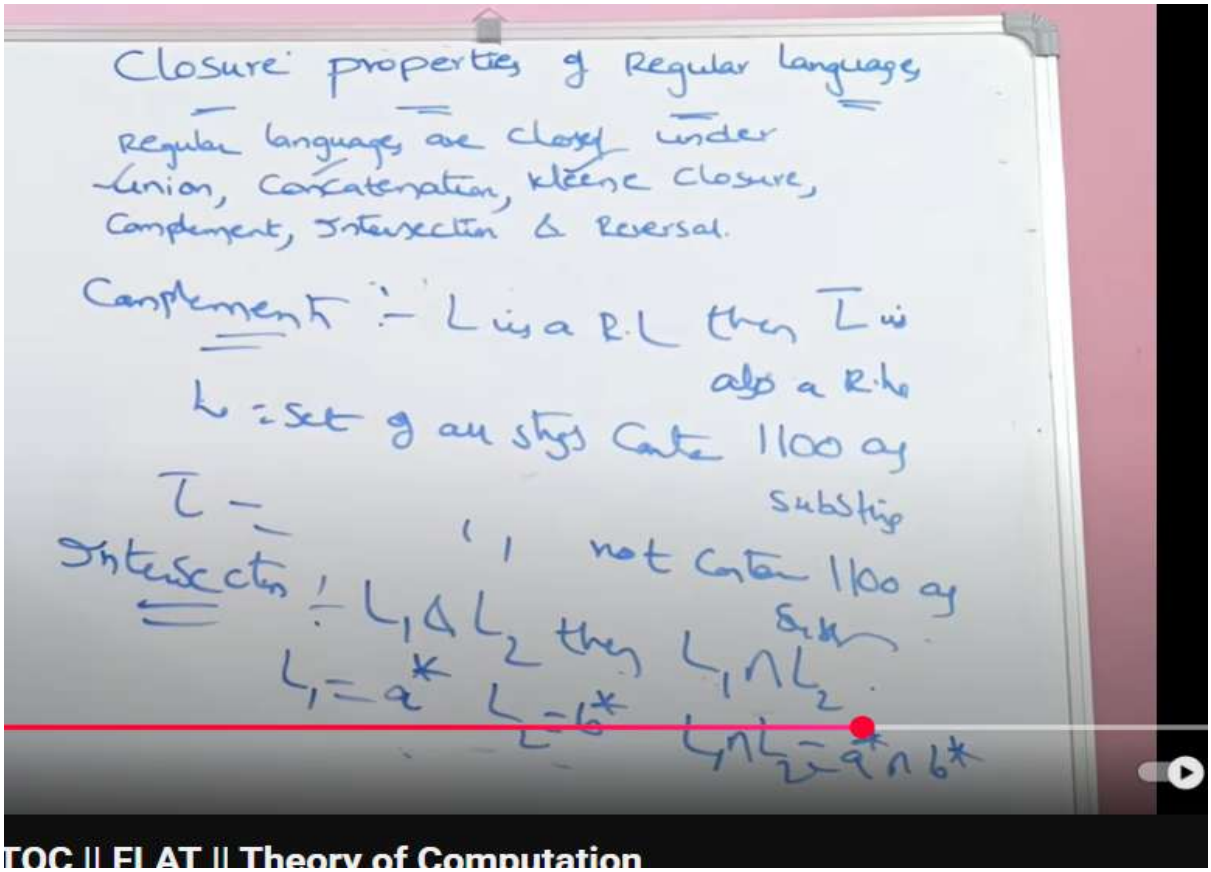


Application of Pumping Lemma

- Proving a Language is Not Regular
- Analyzing Context-Free Languages
- Designing Automata and Grammars
- Verifying Language Properties
- Testing Regular Expression Equivalence
- Algorithm Optimization

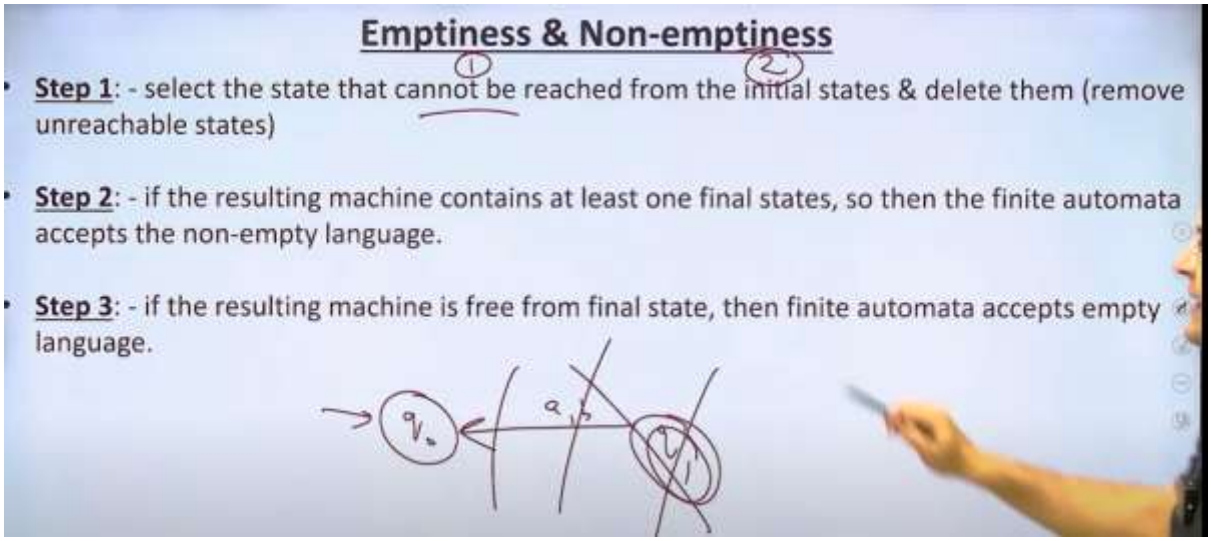
6.Closure Property of Regular Expression





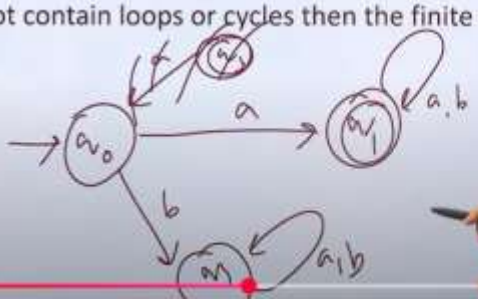
TOC || FLAT || Theory of Computation

7.Decision Tree of Regular Language



Finiteness & Infiniteness

- **Step 1**: - Select the state that cannot be reached from the initial state & delete them (remove unreachable states)
- **Step 2**: - Select the state from which we cannot reach the final state & delete them (remove dead states)
- **Step 3**: - If the resulting machine contains loops or cycles then the finite automata accepts infinite language
- **Step 4**: - If the resulting machine do not contain loops or cycles then the finite automata accepts finite language.

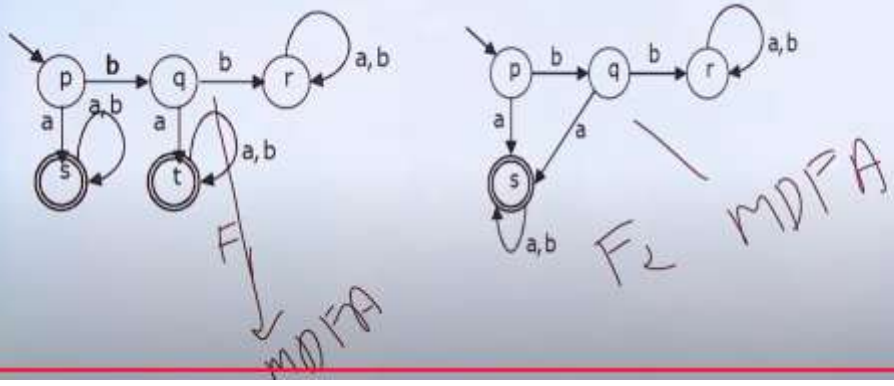


Membership

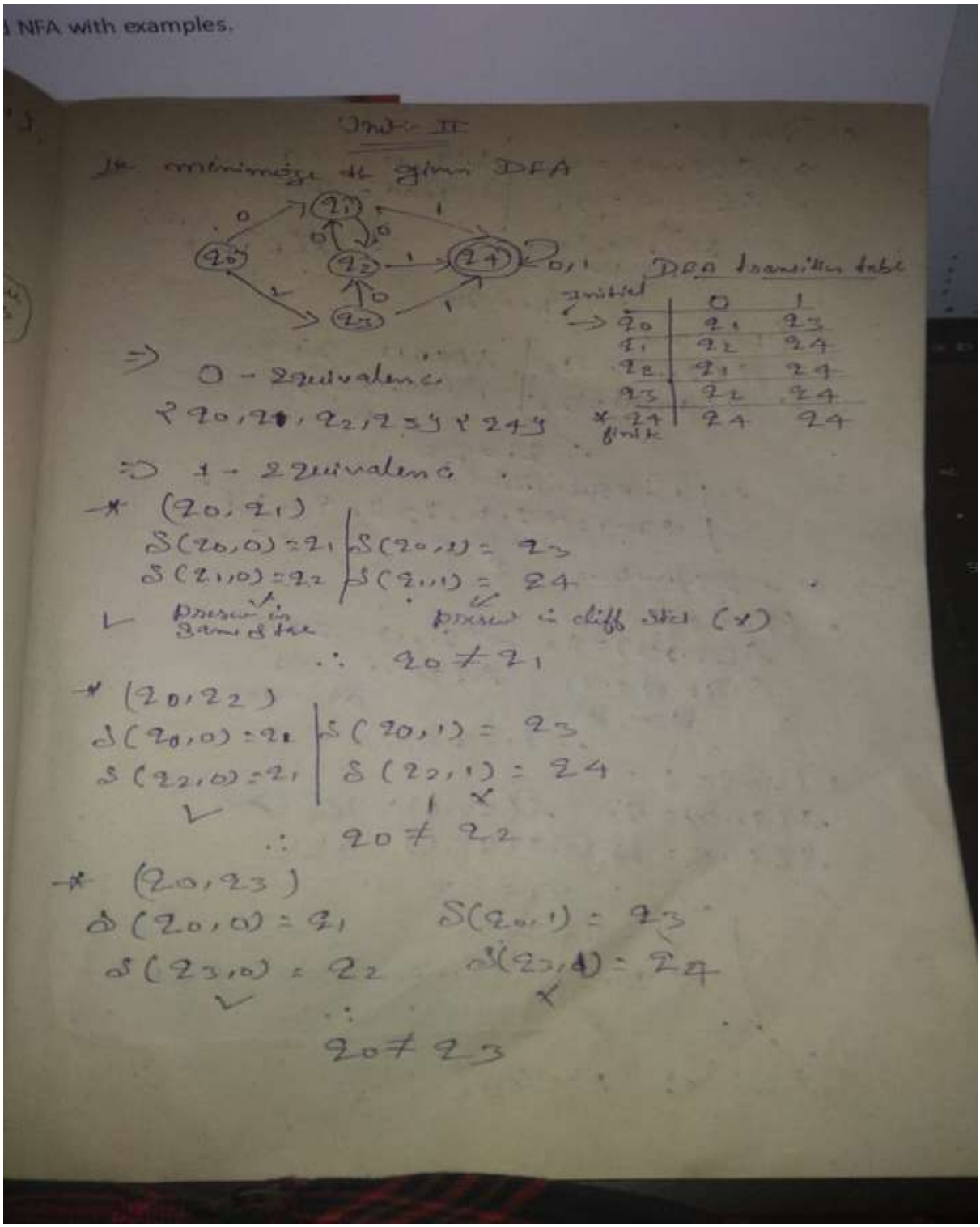
- Membership is a property to verify an arbitrary string is accepted by a finite automaton or not i.e. it is a member of the language or not.
- Let M is a finite automata that accepts some strings over an alphabet, and let 'w' be defined over the alphabet, if there exist a transition path in M, which starts at initial ends in anyone of the final state, then string 'w' is a member of M, otherwise 'w' is member of M.

Equality

- Two finite state automata M_1 & M_2 is said to be equal if and only if, they accept the same language.
- Minimise the finite state automata and and the minimal DFA will be unique.



8. Equivalence and Minimization of Automata.



1 Differentiate DFA and NFA with examples.

* (q_1, q_2)

$\Rightarrow \delta(q_1, 0) = q_2$ | $\delta(q_1, 1) = q_4$

$\delta(q_2, 0) = q_1$ | $\delta(q_2, 1) = q_4$

present in same state 1's ; present in same state 1's

$\therefore q_1 = q_2$

* (q_1, q_3)

$\delta(q_1, 0) = q_2$ | $\delta(q_1, 1) = q_4$

$\delta(q_3, 0) = q_2$ | $\delta(q_3, 1) = q_4$

$\therefore q_1 = q_3$

$\therefore$ 1-equivalence

$\{q_0\}, \{q_1, q_2, q_3\}, \{q_4\}$

$\Rightarrow$ 2-Equivalence

* (q_1, q_2)

$\delta(q_1, 0) = q_2$ $\delta(q_1, 1) = q_4$

$\delta(q_2, 0) = q_1$ $\delta(q_2, 1) = q_4$

Same state ; Same state

$\therefore q_1 = q_2$

* (q_1, q_3)

$\delta(q_1, 0) = q_2$ $\delta(q_1, 1) = q_4$

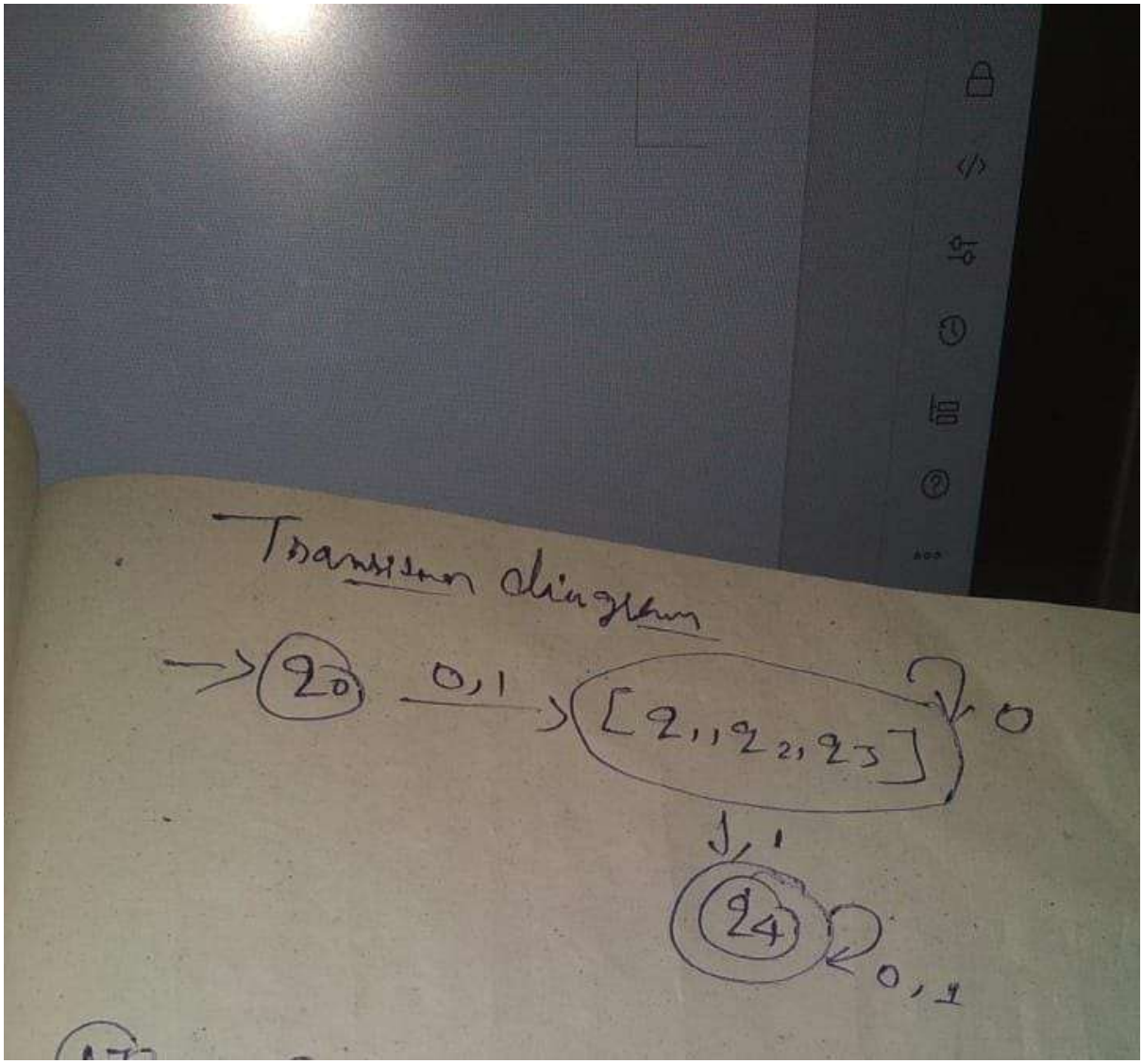
$\delta(q_3, 0) = q_2$ $\delta(q_3, 1) = q_4$

$\therefore q_1 = q_3$

2-Equivalence

$\{q_0\}, \{q_1, q_2, q_3\}, \{q_4\}$

Same as 1-equivalence


Transition diagram
q_0
0,1
$[q_1, q_2, q_3]$
0
1, 1
q_4
0, 1